

Hazardurile în *pipeline*-urile de instrucțiuni

1. Concepte de bază

Există trei cauze majore care generează probleme (induc penalități de performanță) în *pipeline*-urile de instrucțiuni:

1. hazardurile structurale (conflictele la resurse)
2. hazardurile de pe fluxul de control al programului (dependențele procedurale cauzate în special de instrucțiunile *branch*)
3. hazardurile de date (dependențele de date)

Hazardurile structurale sau conflictele la resurse se elimină prin **multiplicarea** resurselor:

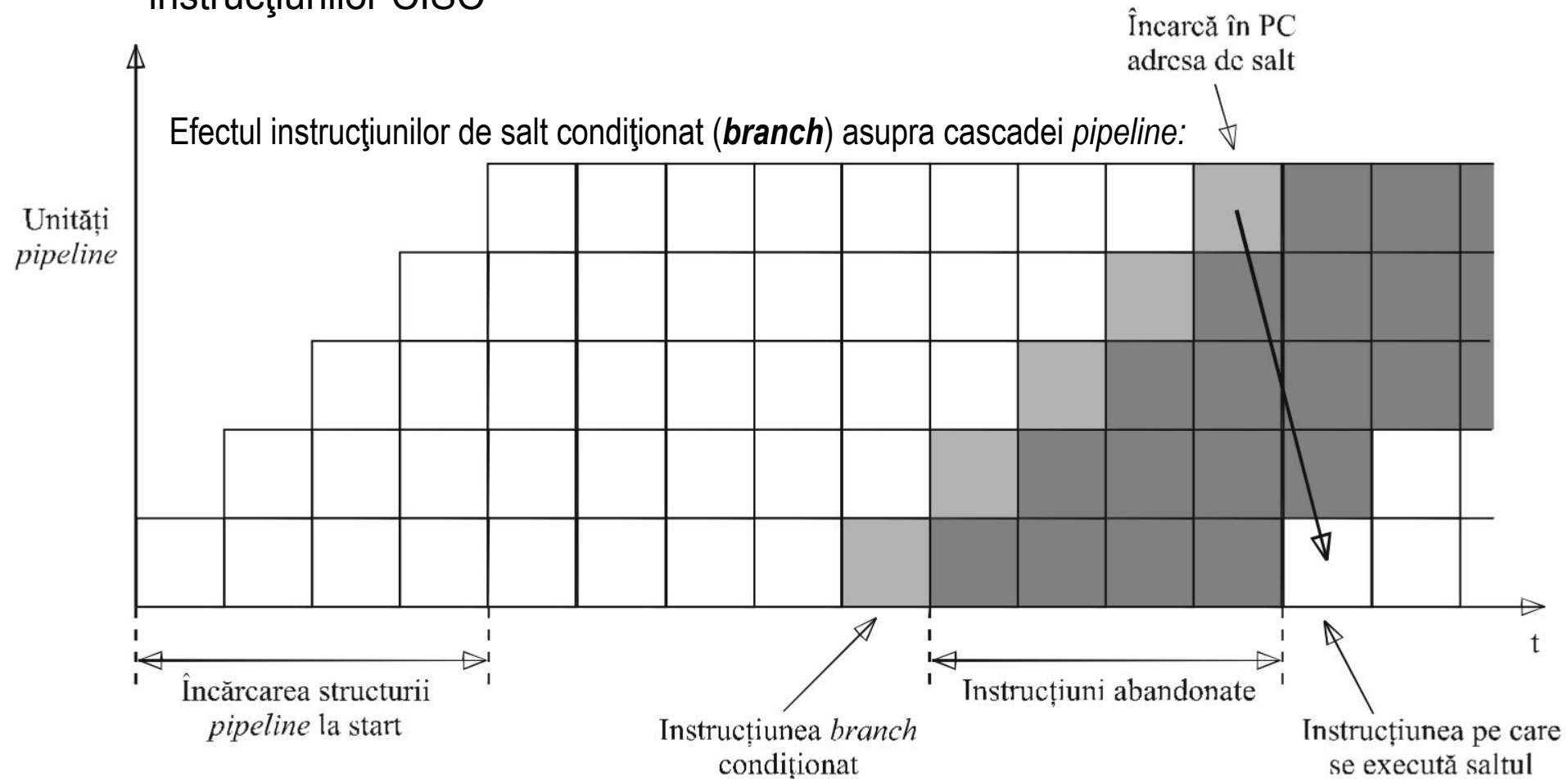
- **mai multe** unități ALU pentru execuția simultană a mai multor instrucțiuni aritmetico-logice în procesoarele superscalare
- **memorii separate** pentru instrucțiuni și respectiv date (arhitectura *Harvard* la nivelul MP) și respectiv **cache-uri separate** pentru instrucțiuni (*I-cache*) și respectiv date (*D-cache*) pentru paralelizarea acceselor de tip *fetch* instrucțiune (lansate de unitatea IF) și respectiv *fetch* operand (lansate de unitatea OF)
- organizarea memoriei în **multiple module** cu adresare întreșută pentru efectuarea mai multor tranzacții simultane cu memoria, etc.)

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

Deși programele sunt scrise sub forma unor secvențe liniare (ordonate) de instrucțiuni, ordinea în care instrucțiunile sunt procesate este necunoscută înaintea execuției:

- dependențele procedurale (*control hazards*) cauzate de instrucțiunile de salt
- dependențele procedurale (*control hazards*) cauzate de lungimea variabilă a instrucțiunilor CISC



Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

Tipic, 10% – 20% din instrucțiunile unui program sunt salturi condiționate (*branch*-uri). Dacă un *pipeline* cu 5 nivele operează cu un tact cu perioada de 10 ns, și doar 10% din instrucțiunile programului sunt *branch*-uri, atunci timpul mediu de procesare a unei instrucțiuni va fi:

$$\frac{9 \cdot 10ns + 1 \cdot 50ns}{10} = 14ns$$

ceea ce indică o reducere a performanțelor ideale cu 40% (de la o instrucțiune per 10 ns la o instrucțiune per 14 ns). Dacă 20% din instrucțiunile programului sunt *branch*-uri, atunci penalitatea va fi de 80%.

Instrucțiunile de salt condiționat sunt utilizate în programare pentru:

- crearea buclelor de program și implementarea condițiilor de ieșire din buclă
- implementarea ieșirilor din buclă în caz de eroare

Din acest motiv, marea majoritate a *branch*-urilor sunt polarizate, fie pe “**taken**” (provoacă salt deoarece foarte frecvent condiția de salt este îndeplinită), fie pe “**not taken**” (nu provoacă salt deoarece foarte frecvent condiția de salt este neîndeplinită).

Informația relativă la polaritatea unui *branch* poate fi utilizată (speculativ) pentru a reduce penalitățile induse de acel *branch*.

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

Exemple de *branch*-uri polarizate:

Pentru implementarea buclei:

```
for (i==0;i<=100;i++) loop_body
```

compilatorul ar putea genera secvența:

```
SUB R1,R1,R1    ; în R1 se încarcă 0
ADD R2,R0,100   ; în R2 se încarcă 100
L2:  BL R2,R1,L1 ; ieșire dacă i > 100
...
Loop_body
...
ADD R1,R1,1     ; incrementare i
JMP L2
L1:
```

unde R1 este alocat contorului de buclă *i* iar R2 conține valoarea de ieșire din buclă.

Instrucțiunea BL R2,R1,L1 este polarizată pe “***not taken***” (atâta timp cât se rămâne în buclă, saltul nu se execută)

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

Exemple de *branch*-uri polarizate:

Pentru implementarea buclei:

```
while (i==j) loop_body
```

compilatorul ar putea genera secvența:

```
L2:  BNE R1,R2,L1
```

```
...
```

```
Loop_body
```

```
...
```

```
JMP L2
```

```
L1:
```

unde R1 este alocat variabilei *i* și R2 variabilei *j*.

Instrucțiunea BNE R1,R2,L1 este polarizată pe “***not taken***” (atâta timp cât se rămâne în buclă, saltul nu se execută)

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

Exemple de *branch*-uri polarizate:

Pentru implementarea buclei:

```
do loop_body while (i==j)
```

compilatorul ar putea genera secvența:

```
L2:      ...  
    Loop_body  
    ...  
    BEQ R1,R2,L2
```

unde R1 este alocat variabilei *i* și R2 variabilei *j*.

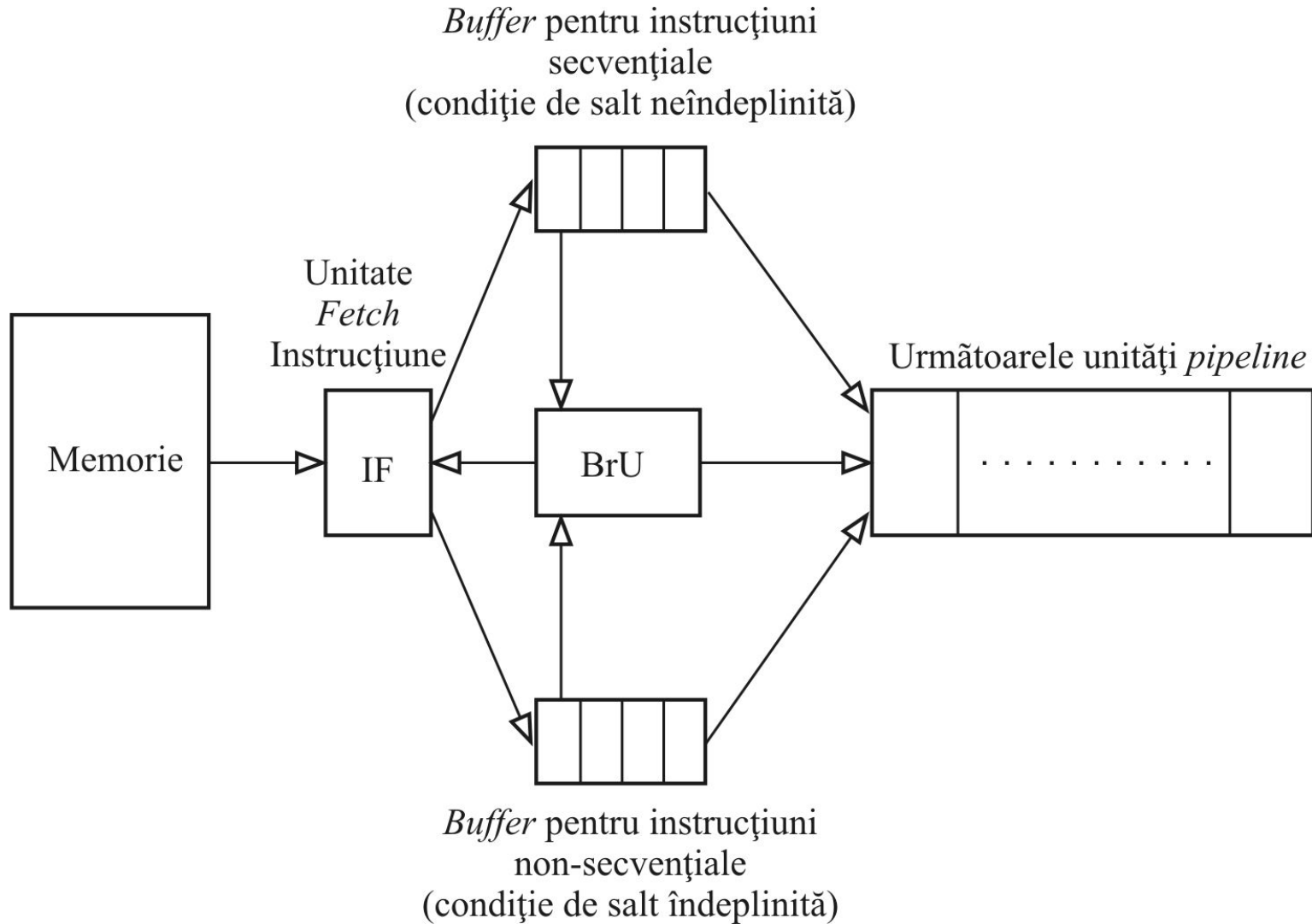
Instrucțiunea `BEQ R1,R2,L2` este polarizată pe “***taken***” (atâta timp cât se rămâne în buclă, saltul se execută).

Un compilator “inteligent” poate deduce polarizarea pentru fiecare *branch* din program și ar putea sugera *hardware*-ului, la momentul execuției respectivului *branch*, ce ramură să urmeze anticipat și speculativ (***taken*** sau ***not taken***). Un astfel de mecanism de predicție implementat în compilator va fi descris la **predicția statică**.

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

A. Tehnica multiplicării *buffer*-lor de *prefetch*:



Două *buffer*-e de *prefetch* pentru reducerea penalităților cauzate de *branch*-urile condiționate

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

A. Tehnica multiplicării *buffere*-lor de *prefetch*:

Fie secvența:

LD R1,A

LD R2,B

LD R3,C

MUL R4,R1,R2

ADD R5,R3,R4

BEQ R5,R0,L1

SUB R5,R5,100

MUL R5,R4,R3

ST [R7],R5

L1: ST 12[R7],R1

ST 16[R7],R2

ST 20[R7],R3

AND R3,R1,R2

Conținut *buffer de prefetch* în momentul în care unitatea BrU detectează în prima locație a *buffer*-ului instrucțiunea BEQ R5,R0,L1. În timp ce structura *pipeline* procesează instrucțiunile LD, MUL și ADD (din fereastră), unitatea BrU calculează adresa de salt L1 și inițiază alimentarea celui alt *buffer de prefetch* cu instrucțiuni începând de la adresa L1 (ST, ST, ST, AND). Apoi BrU se ocupă de evaluarea condiției de salt și imediat după evaluare, selectează doar *buffer*-ul revendicat de valarea (*true/false*) a condiției și descarcă celălalt *buffer*, care va rămâne inactiv până la următorul *branch*.

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

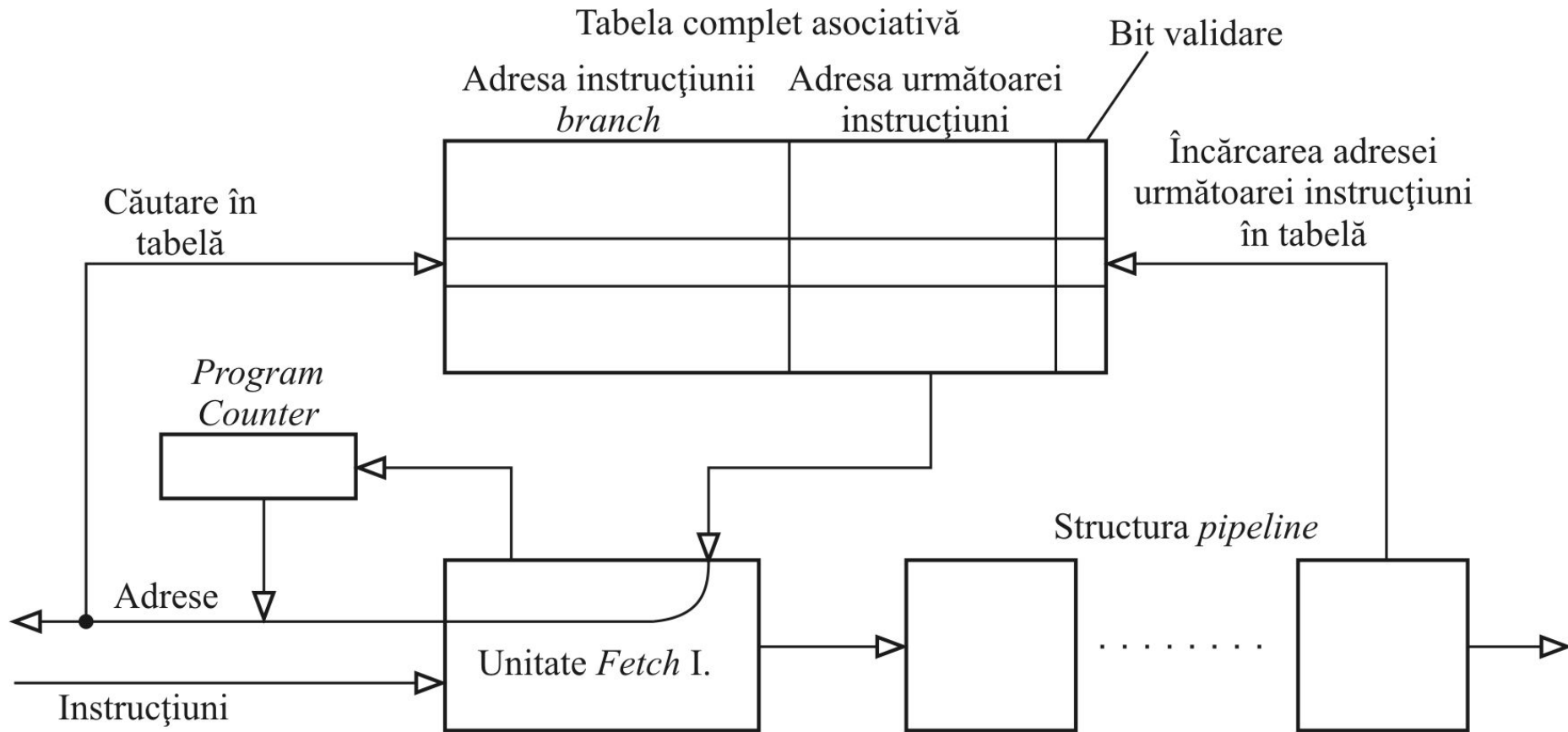
B. Predicția dinamică:

- Se bazează pe **tabele de predicție**, denumite *branch history table* sau *branch target buffer*, implementate în procesor (în *hardware*), având o structură de *cache* specializat în predicția *branch*-urilor.
- La prima execuție a unei instrucțiuni *branch*, acesteia i se alocă o intrare (o locație) în tabelă. În intrare se memorează:
 - în zona TAG: adresa (PC-ul) instrucțiunii *branch*
 - în zona DATA: adresa sau chiar codul următoarei instrucțiuni (care a fost executată după *branch*), informația de predicție și bitul de validare $V=1$ care va certifica faptul că intrarea a fost alocată *branch*-ului aflat în memorie la adresa stocată în zona TAG aferentă acestei intrării.
- Ori de câte ori unitatea IF inițiază *fetch*-ul unei instrucțiuni din memorie, adresa de *fetch* se compară cu toate adresele memorate în zona TAG a tablei de predicție (structură de memorie asociativă, după cum am stabilit la *cache*-urile de date). La *hit* în tabelă pe o intrare cu bitul V pe 1, adresa sau chiar codul următoarei instrucțiuni va fi prelată (preluat) din tabelă. Prin urmare, pentru a reduce din penalitate, următoarea instrucțiune va fi lansată în execuție rapid și speculativ, pe baza informațiilor stocate în tabela de predicție. Dacă predicția se va dovedi corectă, se va procesa fără penalități; dacă se va dovedi greșită, se va reveni în punctul de ramificație (se face *roll-back* anulând toate modificările pe care instrucțiunile speculativ executate le-au făcut în regiștri) și se reporneste din punctul de ramificație pe cealaltă direcție. Se actualizează de asemenea și informația de predicție din tabelă.

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

B. Predicția dinamică:



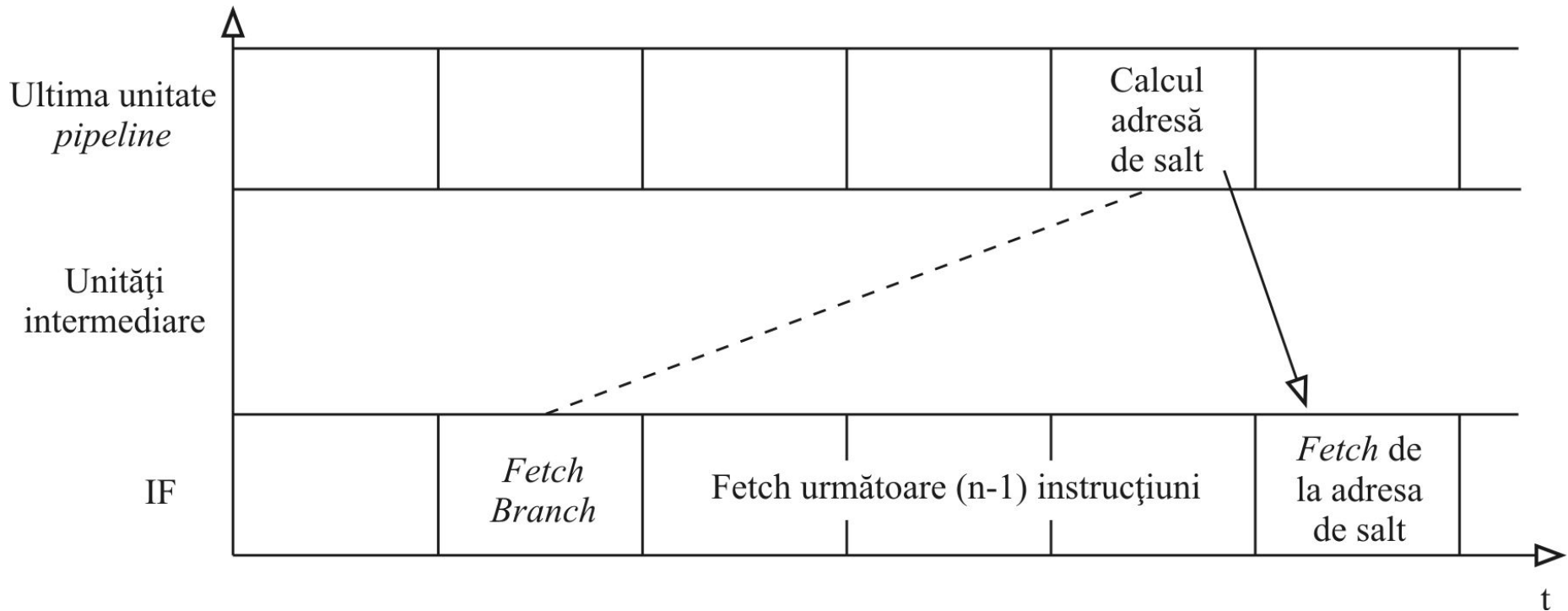
Procesor *pipeline* cu tabelă de predicție

Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

C. *Branch*-urile întârziate :

O instrucțiune *branch* întârziată cu k instrucțiuni (*delayed branch*) este astfel proiectată încât să aibă efect doar după k instrucțiuni (introduce un punct de ramificație întârziat cu k instrucțiuni). Pentru un *pipeline* pe n nivele, instrucțiunea *branch* trebuie astfel proiectată încât să aibă efect doar după $(n - 1)$ instrucțiuni; se poate ajunge astfel la compensarea integrală a penalităților induse de *branch*:



Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

C. *Branch*-urile întârziate :

Cele ($n-1$) instrucțiuni plasate după instrucțiunea *branch* nu trebuie să afecteze condiția de salt testată în cadrul *branch*-ului. Prin urmare, compilatorul trebuie să găsească (în preajma *branch*-ului) $n-1$ instrucțiuni independente și în faza de optimizare să le plaseze după *branch* (reordonarea instrucțiunilor fără a afecta logica programului). Limitările metodei sunt date de dificultatea găsirii unui număr suficient de mare de instrucțiuni utile programului și independente de instrucțiunea *branch*.

De exemplu, în secvența:

```
ADD R2,R3,R4  
SUB R5,R5,1  
BEQ R5,R0,L1
```

.....

L1:

instrucțiunea `ADD R2,R3,R4` este independentă de `BEQ R5,R0,L1` și ar putea fi plasată după aceasta (reordonare care nu afectează logica programului):

```
SUB R5,R5,1  
BEQ R5,R0,L1  
ADD R2,R3,R4
```

.....

L1:

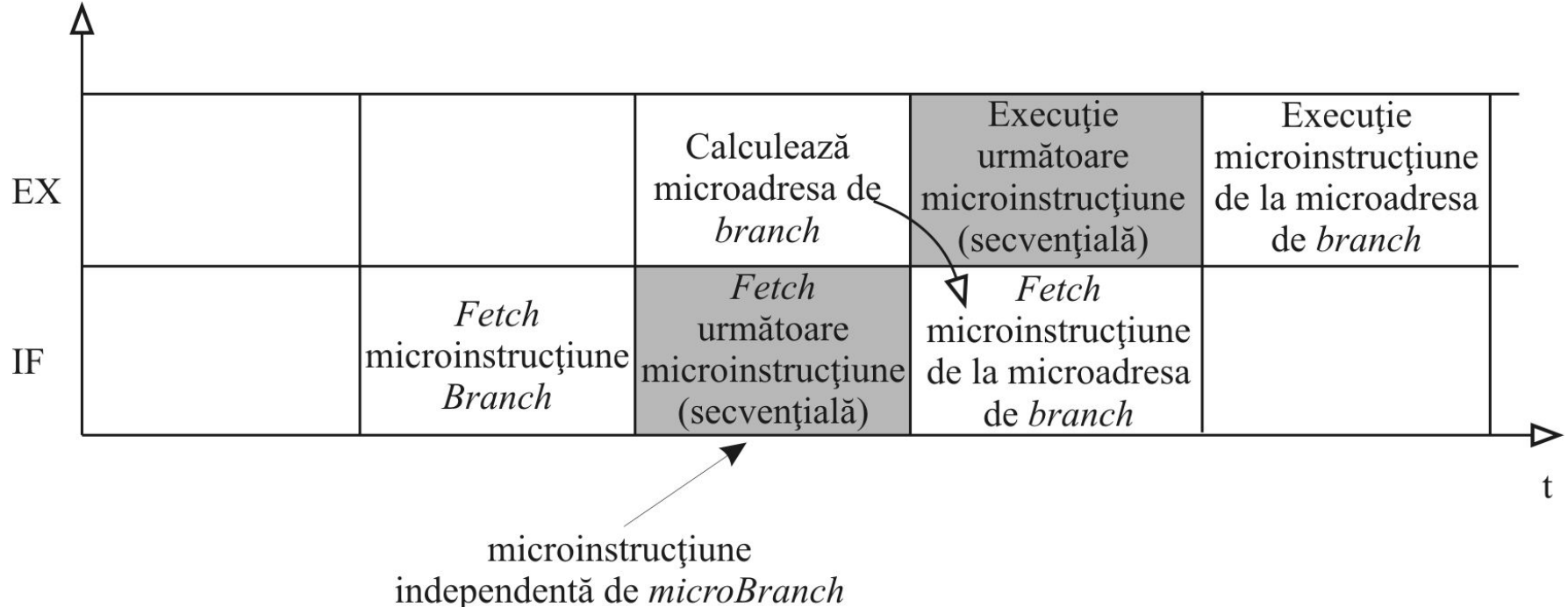
Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

C. *Branch*-urile întârziate :

Setul de instrucțiuni aferent unui procesor poate conține ambele variante de *branch*-uri (întârziate și neîntârziate) sau numai *branch*-uri întârziate. Dacă se prevăd numai *branch*-uri întârziate cu $n-1$ instrucțiuni (n -lungimea cascadei *pipeline*) și dacă nu există suficiente instrucțiuni independente, compilatorul va completa cu NOP-uri pentru a crea un slot complet de $n-1$ instrucțiuni independente, imediat după *branch*.

Tehnica *branch*-urilor întârziate cu o microinstrucțiune este aplicată extensiv la nivelul microcodului (*pipeline* pe două nivele la nivelul microinstrucțiunii !).

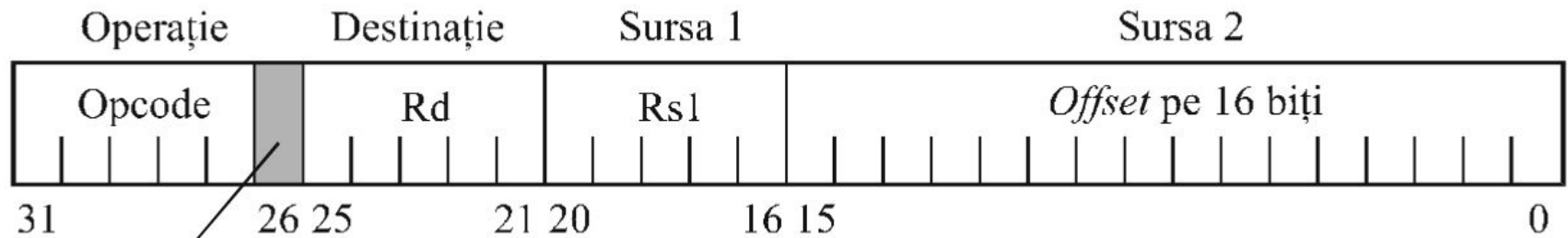


Hazardurile în *pipeline*-urile de instrucțiuni

2. Hazardurile de pe fluxul de control al programului

D. Predicția statică:

Predicția statică se implementează în *software* (compilator). La nivel *hardware* e nevoie doar de implementarea unui bit de predicție în OP-CODE-ul instrucțiunilor *branch* pentru a indica direcția cea mai probabilă pe care se va ieși din punctul de ramificație implementat de respectivul *branch*:



Bit de predicție

1 - selectează adresa de salt pentru *fetch*-ul următoarei instrucțiuni

0 - selectează PC-ul pentru *fetch*-ul următoarei instrucțiuni

Cu ajutorul bitului de predicție se realizează efectiv dublarea setului de instrucțiuni *branch*. De exemplu, instrucțiunea BEQ devine:

BEQT - selectează adresa de salt pentru *fetch*-ul următoarei instrucțiuni (BEQ cu predicție *taken*)

BEQPC - selectează PC-ul pentru *fetch*-ul următoarei instrucțiuni (BEQ cu predicție *not taken*)